

```

"""
bootstrap.py
=====

Rutina de ARRANQUE del proyecto: revisa dependencias, instala lo que falte,
confirma que la instalacion fue exitosa y solo despues deja continuar al
pipeline.

REGLA CRITICA DE ESTE MODULO
-----
Este archivo debe poder importarse ANTES de que exista cualquier dependencia
externa. Por lo tanto solo usa la **libreria estandar** de Python
(`importlib`, `subprocess`, `sys`, `logging`, ...). Ningun `import pandas`
aqui, porque justamente pandas puede no estar instalado todavia.

Comportamiento
-----
1. Para cada dependencia declarada se verifica si el modulo es importable.
2. Si no lo es, se ejecuta `python -m pip install <paquete>` (sin fijar
   version, para obtener la ultima estable disponible).
3. Se invalidan las caches de importacion y se REINTENTA importar.
4. Todo queda registrado en el log del proceso.
5. Si falla una dependencia CRITICA -> se aborta con un mensaje claro.
   Si falla una dependencia OPCIONAL -> se registra el problema y el flujo
   CONTINUA (el pipeline degrada con gracia; p. ej. sin Boruta de libreria
   se usa la implementacion nativa incluida en el proyecto).
"""

from __future__ import annotations

import importlib
import importlib.util
import logging
import subprocess
import sys
from dataclasses import dataclass, field
from typing import Any

LOGGER = logging.getLogger("featsel.bootstrap")

# -----
# Declaracion de dependencias
# -----
@dataclass(frozen=True)
class Dependencia:
    """Describe una dependencia del proyecto.

    Attributes
    -----
    modulo : str
        Nombre con el que se IMPORTA (ej. ``sklearn``).
    paquete : str
        Nombre con el que se INSTALA en PyPI (ej. ``scikit-learn``).
        No siempre coinciden, por eso se declaran por separado.
    critica : bool
        Si ``True`` y no se puede instalar, el pipeline no puede continuar.
    motivo : str
        Para que se usa. Se escribe en el log y en la hoja de parametros.
    """

    modulo: str
    paquete: str
    critica: bool
    motivo: str = ""

#: Dependencias del nucleo: sin ellas el pipeline no tiene sentido.
DEPENDENCIAS_NUCLEO: tuple[Dependencia, ...] = (
    Dependencia("numpy", "numpy", True, "algebra y calculo numerico"),
    Dependencia("pandas", "pandas", True, "manipulacion del panel"),

```

```

Dependencia("scipy", "scipy", True, "estadísticos (chi2, rangos, Spearman)"),
Dependencia("sklearn", "scikit-learn", True, "AUC/Gini, RandomForest para Boruta"),
Dependencia("openpyxl", "openpyxl", True, "escritura y formato del Excel de bitacora"),
)

#: Dependencias de soporte: mejoran el proceso pero admiten fallback.
DEPENDENCIAS_SOPORTE: tuple[Dependencia, ...] = (
    Dependencia("yaml", "PyYAML", False, "lectura de config.yaml"),
    Dependencia("joblib", "joblib", False, "paralelismo de scikit-learn"),
    Dependencia("pyarrow", "pyarrow", False, "lectura de .parquet / .feather"),
)

#: Dependencias de la fase 4 (opcional). Solo se resuelven si usar_boruta=True.
DEPENDENCIAS_BORUTA: tuple[Dependencia, ...] = (
    Dependencia("boruta", "Boruta", False, "algoritmo Boruta (BorutaPy)"),
    Dependencia("shap", "shap", False, "valores SHAP para BorutaShap"),
    Dependencia("BorutaShap", "BorutaShap", False, "variante Boruta basada en SHAP"),
)

# -----
# Reporte de resolucíon
# -----
@dataclass
class ResultadoDependencia:
    """Resultado de resolver una dependencia (queda en la bitacora)."""

    modulo: str
    paquete: str
    critica: bool
    motivo: str
    estado: str = "PENDIENTE" # YA_INSTALADA | INSTALADA_AUTOMATICAMENTE | FALLIDA | OMITIDA
    version: str = ""
    detalle: str = ""

@dataclass
class ReporteBootstrap:
    """Agregado de todos los resultados de la fase de arranque."""

    resultados: list[ResultadoDependencia] = field(default_factory=list)
    instalaciones_automaticas: list[str] = field(default_factory=list)
    fallidas_criticas: list[str] = field(default_factory=list)
    fallidas_opcionales: list[str] = field(default_factory=list)

    @property
    def ok(self) -> bool:
        """El arranque es válido si no fallo ninguna dependencia crítica."""
        return not self.fallidas_criticas

    def a_filas(self) -> list[dict[str, Any]]:
        """Serializa el reporte para volcarlo a la hoja de parámetros."""
        return [
            {
                "modulo": r.modulo,
                "paquete_pypi": r.paquete,
                "critica": int(r.critica),
                "para_que_sirve": r.motivo,
                "estado": r.estado,
                "version_detectada": r.version,
                "detalle": r.detalle,
            }
            for r in self.resultados
        ]

# -----
# Utilidades internas
# -----
def _modulo_disponible(modulo: str) -> bool:
    """`True` si el modulo puede localizarse sin llegar a ejecutarlo."""

```

```

try:
    return importlib.util.find_spec(modulo) is not None
except (ImportError, ValueError, ModuleNotFoundError):
    # find_spec puede reventar si un paquete padre esta roto.
    return False

def _version_de(modulo: str) -> str:
    """Obtiene la version instalada de un modulo ya importable (best effort)."""
    try:
        mod = importlib.import_module(modulo)
        return str(getattr(mod, "__version__", "") or "")
    except Exception: # noqa: BLE001 - version es informativa, nunca bloquea
        return ""

def _asegurar_pip() -> bool:
    """Verifica que `pip` este disponible; intenta repararlo con `ensurepip`."""
    try:
        subprocess.run(
            [sys.executable, "-m", "pip", "--version"],
            check=True,
            capture_output=True,
            timeout=180,
        )
        return True
    except Exception: # noqa: BLE001
        LOGGER.warning("pip no responde; intentando reconstruirlo con ensurepip...")
        try:
            subprocess.run(
                [sys.executable, "-m", "ensurepip", "--upgrade"],
                check=True,
                capture_output=True,
                timeout=300,
            )
            return True
        except Exception as exc: # noqa: BLE001
            LOGGER.error("No fue posible habilitar pip: %s", exc)
            return False

def _pip_install(paquete: str, timeout: int = 900) -> tuple[bool, str]:
    """Instala/actualiza un paquete con pip.

    No se fija version: se solicita la ultima estable publicada en PyPI,
    conforme al requisito del proyecto.
    """
    cmd = [
        sys.executable, "-m", "pip", "install", "--upgrade",
        "--disable-pip-version-check", "--no-input", paquete,
    ]
    LOGGER.info("Instalando dependencia faltante: %s", " ".join(cmd))
    try:
        proc = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
    except subprocess.TimeoutExpired:
        return False, f"timeout de {timeout}s instalando {paquete}"
    except Exception as exc: # noqa: BLE001
        return False, f"excepcion al invocar pip: {exc}"

    if proc.returncode == 0:
        return True, (proc.stdout or "").strip().splitlines()[-1] if proc.stdout else "ok"
    salida = ((proc.stderr or "") + (proc.stdout or "")).strip()
    return False, salida[-500:] # se recorta: los traceback de pip son enormes

# -----
# API publica
# -----
def asegurar_dependencias(
    dependencias: tuple[Dependencia, ...],
    autoinstalar: bool = True,

```

```

    reporte: ReporteBootstrap | None = None,
) -> ReporteBootstrap:
    """Revisa -> instala -> confirma -> continua, para un grupo de dependencias.

Parameters
-----
dependencias
    Grupo a resolver (nucleo, soporte o boruta).
autoinstalar
    Si ``False`` solo verifica y reporta, sin tocar el entorno. Util en
    entornos corporativos sin salida a internet o con el entorno sellado.
reporte
    Reporte acumulado al que se agregan los resultados (permite encadenar
    varias llamadas y exportar todo junto a la bitacora).

Returns
-----
ReporteBootstrap
    Acumulado con el estado de cada dependencia.
"""
reporte = reporte or ReporteBootstrap()
pip_ok: bool | None = None # se evalua perezosamente, solo si hace falta

for dep in dependencias:
    res = ResultadoDependencia(dep.modulo, dep.paquete, dep.critica, dep.motivo)

    # --- Paso 1: ya esta instalada? -----
    if _modulo_disponible(dep.modulo):
        res.estado = "YA_INSTALADA"
        res.version = _version_de(dep.modulo)
        LOGGER.info("[dep] %-12s OK (v%s)", dep.modulo, res.version or "?")
        reporte.resultados.append(res)
        continue

    # --- Paso 2: instalar automaticamente -----
    if not autoinstalar:
        res.estado = "OMITIDA"
        res.detalle = "autoinstalacion desactivada (--sin-autoinstall)"
        LOGGER.warning("[dep] %-12s FALTA y la autoinstalacion esta desactivada", dep.modulo)
        (reporte.fallidas_criticas if dep.critica else reporte.fallidas_opcionales).append(dep.modulo)
        reporte.resultados.append(res)
        continue

    if pip_ok is None:
        pip_ok = _asegurar_pip()
    if not pip_ok:
        res.estado = "FALLIDA"
        res.detalle = "pip no disponible en el interprete actual"
        (reporte.fallidas_criticas if dep.critica else reporte.fallidas_opcionales).append(dep.modulo)
        reporte.resultados.append(res)
        continue

    exito, detalle = _pip_install(dep.paquete)

    # --- Paso 3: confirmar con reintento de importacion -----
    # No basta con que pip devuelva 0: hay que invalidar las caches del
    # sistema de importacion y comprobar que el modulo YA se puede cargar
    # en ESTE proceso. Sin esto, Python seguiria creyendo que no existe.
    importlib.invalidate_caches()
    disponible = _modulo_disponible(dep.modulo)

    if exito and disponible:
        res.estado = "INSTALADA_AUTOMATICAMENTE"
        res.version = _version_de(dep.modulo)
        res.detalle = detalle
        reporte.instalaciones_automaticas.append(f"{dep.paquete}={res.version or '?'}")
        LOGGER.info("[dep] %-12s INSTALADA automaticamente (v%s)", dep.modulo, res.version or "?")
    else:
        res.estado = "FALLIDA"
        res.detalle = detalle if not exito else "pip reporto exito pero el modulo sigue sin importarse"
        nivel = LOGGER.error if dep.critica else LOGGER.warning

```

```

        nivel(
            "[dep] %-12s NO se pudo instalar (%s). %s",
            dep.modulo,
            "CRITICA" if dep.critica else "opcional",
            res.detalle,
        )
        (reporte.fallidas_criticas if dep.critica else reporte.fallidas_opcionales).append(dep.modulo)

    reporte.resultados.append(res)

return reporte

def arrancar(usr_boruta: bool = False, autoinstalar: bool = True) -> ReporteBootstrap:
    """Rutina de arranque completa del proyecto.

    Resuelve nucleo + soporte y, solo si ``usr_boruta`` es ``True``, tambien
    las dependencias de la fase 4. No tiene sentido pagar el costo de instalar
    Boruta/SHAP si el usuario desactivo esa fase.

    Raises
    -----
    RuntimeError
        Si alguna dependencia CRITICA no pudo quedar disponible.
    """
    LOGGER.info("=" * 78)
    LOGGER.info("BOOTSTRAP: verificacion e instalacion automatica de dependencias")
    LOGGER.info("Interprete: %s", sys.executable)
    LOGGER.info("Python      : %s", sys.version.split()[0])
    LOGGER.info("=" * 78)

    reporte = asegurar_dependencias(DEPENDENCIAS_NUCLEO, autoinstalar)
    reporte = asegurar_dependencias(DEPENDENCIAS_SOPORTE, autoinstalar, reporte)

    if usr_boruta:
        LOGGER.info("usr_boruta=True -> resolviendo dependencias de la fase 4")
        reporte = asegurar_dependencias(DEPENDENCIAS_BORUTA, autoinstalar, reporte)
    else:
        LOGGER.info("usr_boruta=False -> se omite la resolucion de Boruta/BorutaShap")
        for dep in DEPENDENCIAS_BORUTA:
            reporte.resultados.append(
                ResultadoDependencia(
                    dep.modulo, dep.paquete, dep.critica, dep.motivo,
                    estado="OMITIDA", detalle="fase 4 desactivada por parametro",
                )
            )

    if reporte.instalaciones_automaticas:
        LOGGER.info(
            "Instalaciones automaticas realizadas en esta corrida: %s",
            ", ".join(reporte.instalaciones_automaticas),
        )
    if reporte.fallidas_opcionales:
        LOGGER.warning(
            "Dependencias OPCIONALES no disponibles (el flujo continua con fallback): %s",
            ", ".join(reporte.fallidas_opcionales),
        )
    if not reporte.ok:
        raise RuntimeError(
            "No se pudieron resolver dependencias CRITICAS: "
            + ", ".join(reporte.fallidas_criticas)
            + ". Revise conectividad de red o permisos de instalacion."
        )

    LOGGER.info("BOOTSTRAP completado. El pipeline puede continuar.")
    return reporte

```